

PC100: Week 1

Lesson 1: Writing Your First Program

The code editor consists of a top panel (for writing code) and a bottom panel (the console, for viewing output). To display a message on the screen, use the `print()` function.

```
print("Hello World")
```

Output: Hello World

In Python, quotation marks denote text (a **string**). The `print()` function removes the quotes when outputting the text.

Lesson 2: Unlocking print() Power-Ups

The `print()` function has several built-in behaviors that make formatting output easier:

- **Multiple Items:** Separate items with a comma to print them with an automatic space.
`print("red", "blue")` → red blue
- **New Lines:** Every individual `print()` statement automatically starts on a new line.
- **Math inside print:** Python evaluates math before printing.
`print(2 + 3)` → 5
- **Concatenation (+):** Glues strings together *without* spaces.
`print("ice" + "cream")` → icecream
- **Repetition (*):** Repeats a string multiple times.
`print("ha" * 3)` → hahaha

Note: You cannot combine text and numbers using the `+` sign (this causes an error). Use a comma instead: `print("Answer:", 42)`.

Why use a comma instead of '+' for mixed types?

Using + to combine a string and a number causes a `TypeError` because Python does not know how to "add" text to a number. A comma is the safest way to print mixed types because Python automatically converts the number to text and adds a space between them for you.

Lesson 3: Creating Art with Code

You can use keyboard symbols (ASCII characters) to draw shapes and pictures in the console. When creating ASCII art, spaces are just as important as the symbols to create the correct layout.

```
print(" * * * ")
print("#####")
```

Lesson 4: Introduction to Variables

A variable is a labeled box in computer memory used to store data. You use the = sign to assign a value to a variable.

```
greeting = "hello"
print(greeting)
```

Variable Naming Rules:

- Use only letters, numbers, or underscores (`_`).
- Cannot start with a number.
- No spaces allowed.
- Case-sensitive (Name is different from name).
- Cannot use Python reserved keywords (like `class`, `if`, `print`).

Common Variable Name Pitfalls

Always avoid using Python's "reserved words" for your variable names. If you name a variable `print`, you will lose the ability to use the `print()` function for the rest of your script because your variable name "shadows" (overwrites) the original function.

Lesson 5: Working with Text

Information stored in a computer has a specific **data type**. Text is referred to as a **string** (or `str`). Strings must be wrapped in either single quotes (`' '`) or double quotes (`" "`), but the beginning and ending quotes must match.

Lesson 6: Working with Numbers

Python treats numbers differently than text. 5 is a number, while `"5"` is a string. There are two primary number buckets:

- **Integer** (`int`): Whole numbers (e.g., 7, -3, 42).
- **Float** (`float`): Decimal numbers (e.g., 7.5, 3.14).

Math operators include `+` (add), `-` (subtract), `*` (multiply), and `/` (divide). *Note:* Division in Python always returns a float, even if it divides evenly (e.g., `10 / 2` results in `5.0`).

Why does `10 / 2` return `5.0`?

In Python, the true division operator (`/`) is designed to handle cases where division might result in a remainder, so it always outputs a `float`. If you absolutely need a whole number result, use the floor division operator (`//`) instead.

Lesson 7: Checking the Facts (Data Types)

To reveal the specific category (data type) of a value, use the `type()` function.

```
print(type(25))      # <class 'int'>
print(type(7.99))    # <class 'float'>
print(type("Bot"))   # <class 'str'>
```

Lesson 8: Type Errors & f-strings

Attempting to use the `+` operator to combine a string and a number results in a **TypeError** (e.g., `"Age: " + 25` will crash the program).

The cleanest way to embed variables inside text is using an **f-string** (formatted string). Place an `f` before the opening quote and put variables inside curly braces `{}`.

```
age = 25
print(f"Age: {age}")
```

Output: Age: 25

Why are f-strings "safer"?

When you use `+` to join values, you have to manually convert every number to a string using `str()`. If you forget even one, your program crashes. f-strings handle this type conversion automatically inside the curly braces, making your code cleaner and much less prone to errors.

Lesson 9: Type Casting

Sometimes you need to convert data from one type to another (e.g., converting text from a website into a mathable number).

- `str()`: Converts a value to a string (text).
- `int()`: Converts a value to a whole number.
- `float()`: Converts a value to a decimal number.

If you attempt to convert text that isn't a number (e.g., `int("hello")`), Python will throw a **ValueError**.

What is a 'ValueError'?

A `ValueError` occurs when the function receives an argument that has the right type but an inappropriate value. For example, `int("apple")` is a string, which is the right type, but the *content* "apple" cannot be logically converted into a whole number.

Lesson 10: Comments & Multi-line Strings

Comments are notes written for humans to read and are completely ignored by Python. They start with a hashtag `#`.

You can create multi-line strings (and f-strings) by wrapping text in triple quotes `"""`. This is highly useful for profile cards or formatting blocks of text.

```
# This is a comment
profile = f"""
Name: {name}
Age: {age}
"""
```

Lessons 11-13: Project - Assemble Your Bot

These lessons combine prior concepts into a complete, sequential script. A well-organized script runs top-to-bottom and utilizes comments to divide the code into logical, readable sections.

```
# --- logo ---
logo = """
    @@@
    %%%
    """
print(logo)

# --- introduction ---
bot_name = "Botty"
print(f"Hi, I'm {bot_name}.")

# --- calculation skill ---
hours = 24
minutes = hours * 60
print(f"A day has {minutes} minutes.")
```

Lesson 14: Summary

By completing this pre-course, you have learned the foundational building blocks of Python programming, including executing code with `print()`, managing data via variables and types (`int`, `float`, `str`), checking types and resolving errors, utilizing math operators, and creating dynamic output with `f-strings`.